ASSIGNMENT 2 – CO2

ARTIFICIAL INTELLIGENCE – CSA17

TASK 1 – GREEDY BEST-FIRST AND A* SEARCH UNDER VARYING MAP CONDITIONS

Source Code Implementation

Listing 1.1: Euclidean and Manhattan heuristics

```
import heapq, math, time, random
from collections import deque

# ---------- heuristics ----------
def euclidean(p, q):
    return math.sqrt((p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)

def manhattan(p, q):
    return abs(p[0] - q[0]) + abs(p[1] - q[1])
```

Listing 1.2: One search routine that implements UCS, GBFS and A*

```
# ---------- search ----------
MOVES = [(-1, 0), (1, 0), (0, -1), (0, 1)]

def neighbours(grid, node):
    r, c = node
    for dr, dc in MOVES:
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]) and grid[nr][nc] == 0:
            yield (nr, nc)

def run_campus_search(grid, start, goal, mode="AStar", h=euclidean, weight=1.0):
    """mode in {'UCS','GBFS','AStar'}; returns metrics dictionary."""
    t0 = time.perf_counter()

    def priority(g, node):
        if mode == "UCS":
            return g
        if mode == "GBFS":
            return h(node, goal)
        return g + weight * h(node, goal)

    def tiebreak(node):
        return h(node, goal) if mode == "AStar" else 0

    g_best, parent, closed = {start: 0}, {start: None}, set()
    tie = 0
    frontier = [(priority(0, start), tiebreak(start), tie, start)]
    expanded, generated, max_frontier = 0, 1, 1
    while frontier:
        max_frontier = max(max_frontier, len(frontier))
        _, _, _, node = heapq.heappop(frontier)
        if node in closed:
            continue
        closed.add(node)
        expanded += 1
        g = g_best[node]
```

```
        if node == goal:
            path, cur = [], node
            while cur is not None:
                path.append(cur)
                cur = parent[cur]
            return {"expanded": expanded, "generated": generated,
                    "max_frontier": max_frontier, "cost": g,
                    "time_ms": (time.perf_counter() - t0) * 1000,
                    "path": path[::-1], "closed": closed}
        for nxt in neighbours(grid, node):
            ng = g + 1
            if mode == "GBFS":
                if nxt in g_best:                    # already generated once
                    continue
            elif nxt in closed or ng >= g_best.get(nxt, float("inf")):
                continue
            g_best[nxt], parent[nxt] = ng, node
            generated += 1
            tie += 1
            heapq.heappush(frontier, (priority(ng, nxt), tiebreak(nxt), tie, nxt))
    return None
```

Listing 1.3: Open, concave-trap, corridor and random map generators

```
# ---------- maps ----------
def open_map(n):
    return [[0] * n for _ in range(n)]

def concave_trap_map(n=30):
    """Start sits inside a U-shaped pocket whose opening faces away from the goal."""
    g = open_map(n)
    for r in range(6, 25):
        g[r][16] = 1                                # base of the U
    for c in range(8, 17):
        g[6][c] = 1                                 # upper arm
        g[24][c] = 1                                # lower arm
    return g, (15, 12), (15, 26)

def corridor_map(n=50, seed=7):
    """Horizontal walls every 5 rows, each with two random doorways."""
    rnd = random.Random(seed)
    g = open_map(n)
    for r in range(5, n - 1, 5):
        for c in range(n):
            g[r][c] = 1
        for c in rnd.sample(range(n), 2):
            g[r][c] = 0
    return g

def random_map(n, density, seed):
    rnd = random.Random(seed)
    g = [[1 if rnd.random() < density else 0 for _ in range(n)] for _ in range(n)]
    g[0][0] = g[n - 1][n - 1] = 0
    return g
```

Listing 1.4: Execution trace of task1_search.py

```
# ---------- heuristic verification against exact h*(n) ----------
def true_cost_to_goal(grid, goal):
    dist, q = {goal: 0}, deque([goal])
    while q:
        cur = q.popleft()
        for nxt in neighbours(grid, cur):
            if nxt not in dist:
                dist[nxt] = dist[cur] + 1
                q.append(nxt)
    return dist

def heuristic_check(grid, goal, h):
    hstar = true_cost_to_goal(grid, goal)
    admissible = all(h(n, goal) <= d + 1e-9 for n, d in hstar.items())
    consistent = all(h(n, goal) <= 1 + h(m, goal) + 1e-9
                     for n in hstar for m in neighbours(grid, n))
    return admissible, consistent, len(hstar)
```

```python
def main():
    maps = {
        "Open Map (20x20)": (open_map(20), (0, 0), (19, 19)),
        "Concave Trap (30x30)": concave_trap_map(30),
        "Corridor Maze (50x50)": (corridor_map(50), (0, 0), (49, 49)),
    }
    print("[EXECUTION TRACE - TASK 1 SEARCH ALGORITHMS]")
    for name, (grid, s, t) in maps.items():
        print(f"--- {name}: start={s} goal={t} ---")
        res = {m: run_campus_search(grid, s, t, m) for m in ("UCS", "GBFS", "AStar")}
        res["AStar-M"] = run_campus_search(grid, s, t, "AStar", h=manhattan)
        opt = res["UCS"]["cost"]
        for m, r in res.items():
            print(f"{m:7s} -> Expanded: {r['expanded']:5d} | Generated: {r['generated']:5d} |
                  f"MaxFrontier: {r['max_frontier']:4d} | Cost: {r['cost']:4d} | "
                  f"Time: {r['time_ms']:7.2f} ms | Optimal: {r['cost'] == opt}")

    print("\n--- Heuristic verification against exact h*(n) ---")
    for name, (grid, s, t) in maps.items():
        for hn, hf in (("Euclidean", euclidean), ("Manhattan", manhattan)):
            a, c, cells = heuristic_check(grid, t, hf)
            print(f"{name:24s} {hn:10s} cells={cells:5d} admissible={a} consistent={c}")

    print("\n--- Seeded random obstacle maps (density 20%) ---")
    for n in (20, 30, 50):
        for seed in range(1, 6):
            grid = random_map(n, 0.20, seed)
            s, t = (0, 0), (n - 1, n - 1)
            if run_campus_search(grid, s, t, "UCS") is None:
                continue                                # goal unreachable -> skip seed
            res = {m: run_campus_search(grid, s, t, m) for m in ("UCS", "GBFS", "AStar")}
            res["AStar-M"] = run_campus_search(grid, s, t, "AStar", h=manhattan)
            print(f"n={n:2d} seed={seed} opt={res['UCS']['cost']:3d} | "
                  f"UCS exp={res['UCS']['expanded']:5d} | "
                  f"GBFS exp={res['GBFS']['expanded']:5d} cost={res['GBFS']['cost']:3d} | "
                  f"A*(Euc) exp={res['AStar']['expanded']:5d} | "
                  f"A*(Man) exp={res['AStar-M']['expanded']:5d}")

    print("\n--- Weighted A* (Manhattan) on Corridor Maze ---")
    grid = maps["Corridor Maze (50x50)"][0]
    for w in (1.0, 1.5, 2.0, 3.0, 5.0):
        r = run_campus_search(grid, (0, 0), (49, 49), "AStar", h=manhattan, weight=w)
        print(f"w={w:3.1f}: expanded={r['expanded']:5d} cost={r['cost']}")

if __name__ == "__main__":
    main()
```

Actual Program Output

```
[EXECUTION TRACE - TASK 1 SEARCH ALGORITHMS]
--- Open Map (20x20): start=(0, 0) goal=(19, 19) ---
UCS      -> Expanded:   400 | Generated:   400 | MaxFrontier:    20 | Cost:    38 |
 Time:     1.16 ms | Optimal: True
GBFS     -> Expanded:    39 | Generated:    76 | MaxFrontier:    38 | Cost:    38 |
 Time:     0.16 ms | Optimal: True
AStar    -> Expanded:   363 | Generated:   400 | MaxFrontier:    38 | Cost:    38 |
 Time:     1.31 ms | Optimal: True
AStar-M -> Expanded:    39 | Generated:    75 | MaxFrontier:    37 | Cost:    38 |
 Time:     0.12 ms | Optimal: True
--- Concave Trap (30x30): start=(15, 12) goal=(15, 26) ---
UCS      -> Expanded:   850 | Generated:   856 | MaxFrontier:    30 | Cost:    44 |
 Time:     2.40 ms | Optimal: True
GBFS     -> Expanded:   209 | Generated:   260 | MaxFrontier:    52 | Cost:    50 |
 Time:     0.41 ms | Optimal: False
AStar    -> Expanded:   590 | Generated:   668 | MaxFrontier:    89 | Cost:    44 |
 Time:     1.53 ms | Optimal: True
AStar-M -> Expanded:   262 | Generated:   315 | MaxFrontier:    54 | Cost:    44 |
 Time:     0.71 ms | Optimal: True
--- Corridor Maze (50x50): start=(0, 0) goal=(49, 49) ---
UCS      -> Expanded:  2068 | Generated:  2068 | MaxFrontier:    23 | Cost:   180 |
 Time:     4.39 ms | Optimal: True
GBFS     -> Expanded:  1261 | Generated:  1380 | MaxFrontier:   120 | Cost:   240 |
 Time:     2.39 ms | Optimal: False
AStar    -> Expanded:  1925 | Generated:  1958 | MaxFrontier:    42 | Cost:   180 |
 Time:     5.14 ms | Optimal: True
AStar-M -> Expanded:  1664 | Generated:  2639 | MaxFrontier:   405 | Cost:   180 |
 Time:     5.04 ms | Optimal: True
```

```
--- Heuristic verification against exact h*(n) ---
Open Map (20x20)          Euclidean  cells=  400 admissible=True consistent=True
Open Map (20x20)          Manhattan  cells=  400 admissible=True consistent=True
Concave Trap (30x30)      Euclidean  cells=  865 admissible=True consistent=True
Concave Trap (30x30)      Manhattan  cells=  865 admissible=True consistent=True
Corridor Maze (50x50)     Euclidean  cells= 2068 admissible=True consistent=True
Corridor Maze (50x50)     Manhattan  cells= 2068 admissible=True consistent=True

--- Seeded random obstacle maps (density 20%) ---
n=20 seed=1 opt= 38 | UCS exp=  320 | GBFS exp=   39 cost= 38 | A*(Euc) exp=  27
5 | A*(Man) exp=   42
n=20 seed=3 opt= 38 | UCS exp=  325 | GBFS exp=   39 cost= 38 | A*(Euc) exp=  28
5 | A*(Man) exp=   41
n=20 seed=4 opt= 38 | UCS exp=  310 | GBFS exp=   40 cost= 38 | A*(Euc) exp=  22
4 | A*(Man) exp=   68
n=30 seed=1 opt= 58 | UCS exp=  731 | GBFS exp=   59 cost= 58 | A*(Euc) exp=  58
8 | A*(Man) exp=   73
n=30 seed=2 opt= 58 | UCS exp=  730 | GBFS exp=   60 cost= 58 | A*(Euc) exp=  58
4 | A*(Man) exp=   82
n=30 seed=3 opt= 58 | UCS exp=  703 | GBFS exp=   61 cost= 58 | A*(Euc) exp=  60
6 | A*(Man) exp=  121
n=30 seed=4 opt= 58 | UCS exp=  691 | GBFS exp=   67 cost= 58 | A*(Euc) exp=  58
1 | A*(Man) exp=   81
n=50 seed=1 opt= 98 | UCS exp= 2012 | GBFS exp=  110 cost=100 | A*(Euc) exp= 176
8 | A*(Man) exp=  184
n=50 seed=2 opt= 98 | UCS exp= 2029 | GBFS exp=  146 cost=108 | A*(Euc) exp= 181
2 | A*(Man) exp=  158
n=50 seed=3 opt= 98 | UCS exp= 2003 | GBFS exp=  104 cost=102 | A*(Euc) exp= 165
3 | A*(Man) exp=  225
n=50 seed=4 opt= 98 | UCS exp= 2001 | GBFS exp=  103 cost=100 | A*(Euc) exp= 175
9 | A*(Man) exp=  386
n=50 seed=5 opt= 98 | UCS exp= 1990 | GBFS exp=  101 cost= 98 | A*(Euc) exp= 179
4 | A*(Man) exp=  232
```

```
--- Weighted A* (Manhattan) on Corridor Maze ---
w=1.0: expanded= 1664 cost=180
w=1.5: expanded= 1576 cost=180
w=2.0: expanded= 1495 cost=180
w=3.0: expanded= 1341 cost=180
w=5.0: expanded= 1245 cost=180
```

TASK 2 – ONLINE SEARCH AGENT IN AN UNMAPPED FACILITY

Source Code Implementation

Listing 2.1: Facility generator, local sensor and offline optimum

```
# ============ CONFIG (edit these) ============
SIZES      = [10, 15, 20]        # grid sizes n x n
DENSITIES  = [0.2, 0.3]          # wall density
SEEDS      = [1, 2, 3]           # random seeds per setting
LRTA_TRIALS = 15                 # missions for LRTA*
# =============================================

import random
from collections import deque

DIRS = [(-1, 0), (0, -1), (1, 0), (0, 1)]   # fixed action order: up, left, down, right

# ---------- Listing 2.1 ----------
def make_world(n, density, seed, blocked=False):
    """Random facility; wall = 1.  If blocked=True the goal is sealed inside walls."""
    rnd = random.Random(seed)
    g = [[1 if rnd.random() < density else 0 for _ in range(n)] for _ in range(n)]
    g[0][0] = g[n - 1][n - 1] = 0
    if blocked:
        r, c = n - 1, n - 1
        for dr, dc in ((-1, 0), (0, -1)):
            if 0 <= r + dr < n and 0 <= c + dc < n:
                g[r + dr][c + dc] = 1
    return g

def free_nbrs(grid, cell):
    """Local sensor (radius 1): the rover only sees its 4 adjacent cells."""
    r, c = cell
    n = len(grid)
    return [(r + dr, c + dc) for dr, dc in DIRS
            if 0 <= r + dr < n and 0 <= c + dc < n and grid[r + dr][c + dc] == 0]

def offline_optimal(grid, start, goal):
    """Optimal cost with full map knowledge (BFS) -> denominator of C_r."""
    dist, q = {start: 0}, deque([start])
    while q:
        cur = q.popleft()
        if cur == goal:
            return dist[cur]
        for nb in free_nbrs(grid, cur):
            if nb not in dist:
                dist[nb] = dist[cur] + 1
                q.append(nb)
    return float("inf")
```

Listing 2.2: Online DFS agent and the simulation loop

```
class OnlineDFSAgent:
    """Online DFS (Russell & Norvig ONLINE-DFS-AGENT) with explicit backtracking."""
    def __init__(self, start, order="fixed", goal=None):
        self.current = start
        self.untried = {}
        self.unbacktracked = []
        self.visited = {start}
        self.order = order
        self.goal = goal
    def select_action(self, neighbours):
        cell = self.current
        if cell not in self.untried:
            nb = list(neighbours)
            if self.order == "greedy" and self.goal is not None:
                nb.sort(key=lambda p: abs(p[0] - self.goal[0]) + abs(p[1] - self.goal[1]))
            self.untried[cell] = [p for p in nb if p not in self.visited]
        while self.untried[cell] and self.untried[cell][0] in self.visited:
            self.untried[cell].pop(0)
        if self.untried[cell]:
            nxt = self.untried[cell].pop(0)
            self.unbacktracked.append(cell)
            self.visited.add(nxt)
            self.current = nxt
            return nxt
        if self.unbacktracked:
            nxt = self.unbacktracked.pop()
            self.current = nxt
            return nxt
        return None
def run_online_dfs(grid, start, goal, order="fixed", max_steps=200000):
    agent = OnlineDFSAgent(start, order, goal)
    moves, seen_count, revisits = 0, {start: 1}, 0
    while agent.current != goal and moves < max_steps:
        nxt = agent.select_action(free_nbrs(grid, agent.current))
        if nxt is None:
            break
        moves += 1
        if nxt in seen_count:
            revisits += 1
        seen_count[nxt] = seen_count.get(nxt, 0) + 1
    found = agent.current == goal
    opt = offline_optimal(grid, start, goal)
    return {"moves": moves, "revisits": revisits, "distinct": len(seen_count),
            "found": found, "opt": opt,
            "cr": (moves / opt) if found and opt not in (0, float("inf")) else float("inf"),
            "outcome": "Goal Found" if found else "Safe Exit (No Solution)"}
```

Listing 2.3: Learning Real-Time A* over repeated missions on a static map

```
# ---------- Listing 2.3 ----------
def run_lrta_trials(grid, start, goal, trials=15):
    """LRTA* on a static map: H(s) persists between missions."""
    H = {}
    h0 = lambda p: abs(p[0] - goal[0]) + abs(p[1] - goal[1])
    steps_per_trial = []
    for _ in range(trials):
        cur, steps = start, 0
        while cur != goal and steps < 20000:
            nbs = free_nbrs(grid, cur)
            if not nbs:                        # trapped start: avoid min() on empty
                break
            costs = {nb: 1 + H.get(nb, h0(nb)) for nb in nbs}
            best = min(costs, key=costs.get)
            H[cur] = costs[best]
            cur, steps = best, steps + 1
        steps_per_trial.append(steps)
    return steps_per_trial

# ---------- Experiment driver ----------
def experiment_dfs():
    print("\nTable A: Online DFS (fixed vs greedy order)")
    print(f"{'n':>3} {'dens':>5} {'seed':>4} {'order':>7} {'moves':>6} {'revis':>6} "
          f"{'dist':>5} {'opt':>5} {'C_r':>6}  outcome")
    for n in SIZES:
        for d in DENSITIES:
            for s in SEEDS:
                grid = make_world(n, d, s)
                for order in ("fixed", "greedy"):
                    r = run_online_dfs(grid, (0, 0), (n - 1, n - 1), order)
                    cr = f"{r['cr']:.2f}" if r["cr"] != float("inf") else "inf"
                    print(f"{n:>3} {d:>5} {s:>4} {order:>7} {r['moves']:>6} {r['revisits']:>
                          f"{r['distinct']:>5} {r['opt']:>5} {cr:>6}  {r['outcome']}")

def experiment_blocked():
    print("\nTable B: Unsolvable (goal sealed) -> safe exit")
    for n in SIZES:
        grid = make_world(n, 0.2, SEEDS[0], blocked=True)
        r = run_online_dfs(grid, (0, 0), (n - 1, n - 1))
        print(f"n={n}: moves={r['moves']}, found={r['found']}, outcome={r['outcome']}")
```

```
            if offline_optimal(grid, (0, 0), (n - 1, n - 1)) == float("inf"):
                print(f"n={n} seed={s}: unsolvable, skipped")
                continue
            steps = run_lrta_trials(grid, (0, 0), (n - 1, n - 1), LRTA_TRIALS)
            print(f"n={n} seed={s}: {steps}")

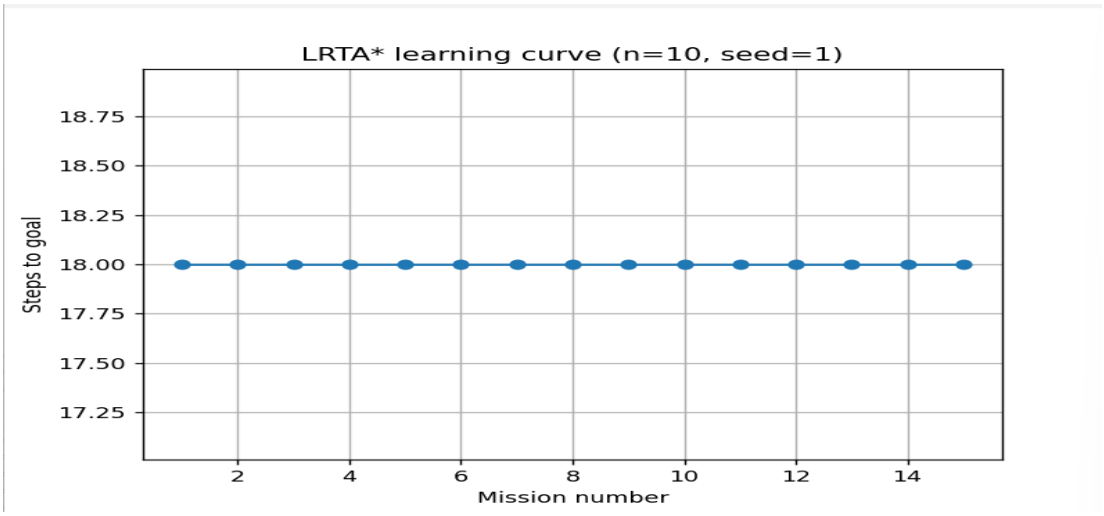
def plot_lrta(n=SIZES[0], seed=SEEDS[0]):
    import matplotlib.pyplot as plt
    grid = make_world(n, DENSITIES[0], seed)
    steps = run_lrta_trials(grid, (0, 0), (n - 1, n - 1), LRTA_TRIALS)
    plt.plot(range(1, len(steps) + 1), steps, marker="o")
    plt.xlabel("Mission number"); plt.ylabel("Steps to goal")
    plt.title(f"LRTA* learning curve (n={n}, seed={seed})")
    plt.grid(True); plt.show()

if __name__ == "__main__":
    experiment_dfs()
    experiment_blocked()
    experiment_lrta()
    plot_lrta()
```

Actual Program Output

```
Table B: Unsolvable (goal sealed) -> safe exit
n=10: moves=162, found=False, outcome=Safe Exit (No Solution)
n=15: moves=358, found=False, outcome=Safe Exit (No Solution)
n=20: moves=634, found=False, outcome=Safe Exit (No Solution)

Table C: LRTA* steps per mission (static map)
n=10 seed=1: [18, 18, 18, 18, 18, 18, 18, 18, 18, 18, 18, 18, 18, 18]
n=10 seed=2: unsolvable, skipped
n=10 seed=3: [20, 18, 18, 18, 18, 18, 18, 18, 18, 18, 18, 18, 18, 18]
n=15 seed=1: [32, 34, 32, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28]
n=15 seed=2: [42, 36, 34, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28]
n=15 seed=3: [30, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28, 28]
n=20 seed=1: [40, 40, 40, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38]
n=20 seed=2: unsolvable, skipped
n=20 seed=3: [42, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38, 38]
```



TASK 3 – ENGINEER THE BEHAVIOUR OF ALPHA-BETA PRUNING

Source Code Implementation

Listing 3.1: Node class, random tree builder and move-ordering routine

```
BRANCHING = [2, 3]            # b values
DEPTHS    = [4, 6, 8]         # tree depths (keep b**depth manageable)
SEEDS     = [1, 2, 3]         # seeds for leaf utilities / shuffling
MODES     = ["best", "random", "worst"]
# ============================================
import random
# ---------- Listing 3.1 ----------
class Node:
    __slots__ = ("value", "children")
    def __init__(self, value=None, children=None):
        self.value, self.children = value, children or []
    def is_leaf(self):
        return not self.children

def build_random_tree(depth, b, rnd):
    """Complete b-ary tree whose leaves hold distinct random utilities."""
    leaves = rnd.sample(range(1, 10 * b ** depth), b ** depth)
    it = iter(leaves)
    def rec(d):
        if d == 0:
            return Node(next(it))
        return Node(None, [rec(d - 1) for _ in range(b)])
    return rec(depth)

def minimax_value(node, is_max):
    if node.is_leaf():
        return node.value
    vals = [minimax_value(c, not is_max) for c in node.children]
    return max(vals) if is_max else min(vals)

def order_tree(node, is_max, mode, rnd):
    """Re-order children: 'best' = strongest move first, 'worst' = weakest first, 'random'."""
    if node.is_leaf():
        return
    for c in node.children:
        order_tree(c, not is_max, mode, rnd)
    if mode == "random":
        rnd.shuffle(node.children)
        return
    keyed = [(minimax_value(c, not is_max), c) for c in node.children]
    best_first_desc = is_max                  # MAX wants high first; MIN wants low first
    keyed.sort(key=lambda t: t[0],
               reverse=best_first_desc if mode == "best" else not best_first_desc)
    node.children = [c for _, c in keyed]
```

Listing 3.2: Instrumented Minimax and Alpha-Beta search

```
# ---------- Listing 3.2 ----------
def minimax_count(node, is_max, counters):
    counters["visited"] += 1
    if node.is_leaf():
        counters["leaves"] += 1
        return node.value
    vals = [minimax_count(c, not is_max, counters) for c in node.children]
    return max(vals) if is_max else min(vals)
def alpha_beta_search(node, depth, alpha, beta, is_max, counters):
    counters["visited"] += 1
    if depth == 0 or node.is_leaf():
        counters["leaves"] += 1
        return node.value
    if is_max:
        best = -float("inf")
        for child in node.children:
            best = max(best, alpha_beta_search(child, depth - 1, alpha, beta, False, counters))
            alpha = max(alpha, best)
            if beta <= alpha:
                counters["cutoffs"] += 1
                break
        return best
    best = float("inf")
    for child in node.children:
        best = min(best, alpha_beta_search(child, depth - 1, alpha, beta, True, counters))
        beta = min(beta, best)
        if beta <= alpha:
            counters["cutoffs"] += 1
            break
    return best
def new_counters():
    return {"visited": 0, "leaves": 0, "cutoffs": 0}
def run_experiment():
    print(f"{'b':>2} {'d':>2} {'seed':>4} {'mode':>7} | {'MM nodes':>9} {'AB nodes':>9} "
          f"{'AB leaves':>9} {'cutoffs':>8} {'saved%':>7} {'same value':>10}")
    summary = {}
    for b in BRANCHING:
        for d in DEPTHS:
            for s in SEEDS:
                for mode in MODES:
                    tree = build_random_tree(d, b, random.Random(s))
                    order_tree(tree, True, mode, random.Random(s + 1000))
                    c_mm = new_counters()
                    v_mm = minimax_count(tree, True, c_mm)
                    c_ab = new_counters()
                    v_ab = alpha_beta_search(tree, d, -float("inf"), float("inf"), True, c_ab)
```

```
                saved = 100 * (1 - c_ab["visited"] / c_mm["visited"])
                print(f"{b:>2} {d:>2} {s:>4} {mode:>7} | {c_mm['visited']:>9} {c_ab['visited']:>9} "
                      f"{c_ab['leaves']:>9} {c_ab['cutoffs']:>8} {saved:>7.1f} {str(v_mm == v_ab):>10}")
                summary.setdefault((b, d, mode), []).append(c_ab["leaves"])
    print("\nAverage Alpha-Beta leaves evaluated (over seeds) vs theory")
    print(f"{'b':>2} {'d':>2} {'mode':>7} {'avg leaves':>11} {'b^d (minimax)':>14} {'b^(d/2) ideal':>14}")
    for (b, d, mode), v in summary.items():
        print(f"{b:>2} {d:>2} {mode:>7} {sum(v)/len(v):>11.1f} {b**d:>14} {b**(d/2):>14.1f}")

def plot_results(b=BRANCHING[0], seed=SEEDS[0]):
    import matplotlib.pyplot as plt
    width = 0.25
    xs = range(len(DEPTHS))
    for i, mode in enumerate(MODES):
        ys = []
        for d in DEPTHS:
            tree = build_random_tree(d, b, random.Random(seed))
            order_tree(tree, True, mode, random.Random(seed + 1000))
            c = new_counters()
            alpha_beta_search(tree, d, -float("inf"), float("inf"), True, c)
            ys.append(c["visited"])
        plt.bar([x + i * width for x in xs], ys, width, label=mode)
    plt.xticks([x + width for x in xs], [f"d={d}" for d in DEPTHS])
    plt.ylabel("Nodes visited"); plt.title(f"Alpha-Beta by move ordering (b={b})")
    plt.legend(); plt.show()

if __name__ == "__main__":
    run_experiment()
    plot_results()
```

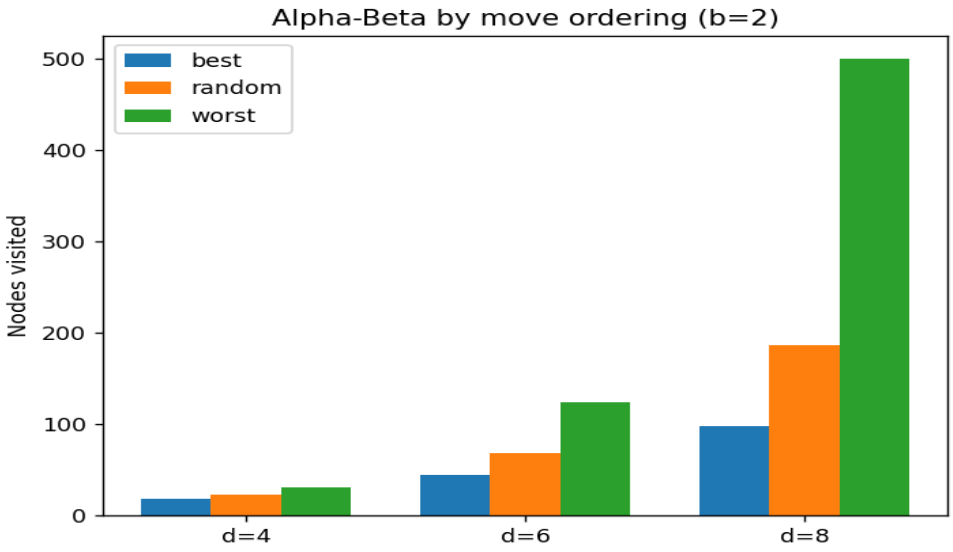
Actual Program Output

b	d	seed	mode	MM nodes	AB nodes	AB leaves	cutoffs	saved%	same value
2	4	1	best	31	18	7	5	41.9	True
2	4	1	random	31	23	11	4	25.8	True
2	4	1	worst	31	31	16	2	0.0	True
2	4	2	best	31	18	7	5	41.9	True
2	4	2	random	31	26	12	5	16.1	True
2	4	2	worst	31	31	16	1	0.0	True
2	4	3	best	31	18	7	5	41.9	True
2	4	3	random	31	20	9	4	35.5	True
2	4	3	worst	31	31	16	0	0.0	True
2	6	1	best	127	44	15	15	65.4	True
2	6	1	random	127	68	29	15	46.5	True
2	6	1	worst	127	124	62	8	2.4	True
2	6	2	best	127	44	15	15	65.4	True
2	6	2	random	127	100	44	18	21.3	True
2	6	2	worst	127	126	63	10	0.8	True
2	6	3	best	127	44	15	15	65.4	True
2	6	3	random	127	84	36	16	33.9	True
2	6	3	worst	127	122	60	12	3.9	True
2	8	1	best	511	98	31	37	80.8	True
2	8	1	random	511	186	77	49	63.6	True
2	8	1	worst	511	500	247	77	2.2	True
2	8	2	best	511	98	31	37	80.8	True
2	8	2	random	511	264	113	59	48.3	True
2	8	2	worst	511	473	232	68	7.4	True
2	8	3	best	511	98	31	37	80.8	True
2	8	3	random	511	194	80	53	62.0	True
2	8	3	worst	511	483	235	81	5.5	True
3	4	1	best	121	37	17	12	69.4	True
3	4	1	random	121	83	49	14	31.4	True
3	4	1	worst	121	120	80	7	0.8	True
3	4	2	best	121	37	17	12	69.4	True
3	4	2	random	121	59	32	13	51.2	True
3	4	2	worst	121	119	79	6	1.7	True
3	4	3	best	121	37	17	12	69.4	True
3	4	3	random	121	71	41	15	41.3	True
3	4	3	worst	121	118	78	7	2.5	True

```
 3  6    1    best |      1093       125        53        46    88.6       True
 3  6    1  random |      1093       377       214        89    65.5       True
 3  6    1   worst |      1093      1004       653       148     8.1       True
 3  6    2    best |      1093       125        53        46    88.6       True
 3  6    2  random |      1093       326       180        75    70.2       True
 3  6    2   worst |      1093      1043       685       116     4.6       True
 3  6    3    best |      1093       125        53        46    88.6       True
 3  6    3  random |      1093       324       189        73    70.4       True
 3  6    3   worst |      1093      1005       655       142     8.1       True
 3  8    1    best |      9841       393       161       152    96.0       True
 3  8    1  random |      9841      2155      1200       562    78.1       True
 3  8    1   worst |      9841      7975      5138      1568    19.0       True
 3  8    2    best |      9841       393       161       152    96.0       True
 3  8    2  random |      9841      2589      1427       716    73.7       True
 3  8    2   worst |      9841      7924      5103      1600    19.5       True
 3  8    3    best |      9841       393       161       152    96.0       True
 3  8    3  random |      9841      2236      1239       604    77.3       True
 3  8    3   worst |      9841      7645      4906      1582    22.3       True
```

Average Alpha-Beta leaves evaluated (over seeds) vs theory

b	d	mode	avg leaves	b^d (minimax)	b^(d/2) ideal
2	4	best	7.0	16	4.0
2	4	random	10.7	16	4.0
2	4	worst	16.0	16	4.0
2	6	best	15.0	64	8.0
2	6	random	36.3	64	8.0
2	6	worst	61.7	64	8.0
2	8	best	31.0	256	16.0
2	8	random	90.0	256	16.0
2	8	worst	238.0	256	16.0
3	4	best	17.0	81	9.0
3	4	random	40.7	81	9.0
3	4	worst	79.0	81	9.0
3	6	best	53.0	729	27.0
3	6	random	194.3	729	27.0
3	6	worst	664.3	729	27.0
3	8	best	161.0	6561	81.0
3	8	random	1288.7	6561	81.0
3	8	worst	5049.0	6561	81.0



TASK 4 – MINIMAX AND THE COST OF EXAMINING EVERY POSSIBILITY

Source Code Implementation

Listing 4.1: Plain Minimax with node and evaluation counters

```
# ============ CONFIG (edit these) ============
B               = 4                         # branching factor of synthetic tree
SYN_DEPTHS      = [2, 4, 6, 8, 10]          # depths for the growth experiment
GAME_DEPTHS     = [1, 2, 4, 6, 8]           # engine depths vs random opponent
GAMES           = {1: 40, 2: 40, 4: 40, 6: 40, 8: 12}   # games per depth (even numbers)
DUP_DEPTHS      = [2, 4, 6, 8]              # depths for duplicate-position study
SEED            = 42
# ============================================

import random
import time

# ---------- Connect-3 constants ----------
ROWS, COLS, K = 4, 4, 3

# ---------- Listing 4.1 ----------
def plain_minimax(state, depth, is_max, counters):
    counters["nodes_expanded"] += 1
    if depth == 0 or state.is_terminal():
        counters["eval_calls"] += 1
        return state.evaluate()
    if is_max:
        best = -float("inf")
        for child in state.get_children():
            best = max(best, plain_minimax(child, depth - 1, False, counters))
        return best
    best = float("inf")
    for child in state.get_children():
        best = min(best, plain_minimax(child, depth - 1, True, counters))
    return best
```

Listing 4.2: Synthetic game tree with branching factor b

```
# ---------- Listing 4.2 ----------
class SyntheticState:
    """Uniform game tree with branching factor b: every state always has b children."""
    __slots__ = ("ident", "b")
    def __init__(self, ident=0, b=4):
        self.ident, self.b = ident, b
    def get_children(self):
        return [SyntheticState(self.ident * self.b + i + 1, self.b) for i in range(self.b)]
    def is_terminal(self):
        return False
    def evaluate(self):
        return (self.ident * 2654435761) % 1009
```

Listing 4.3: Connect-3 (4 x 4) game with immutable states and a heuristic evaluation

```
def _windows():
    wins = []
    for r in range(ROWS):
        for c in range(COLS):
            for dr, dc in ((0, 1), (1, 0), (1, 1), (1, -1)):
                cells = [(r + i * dr, c + i * dc) for i in range(K)]
                if all(0 <= rr < ROWS and 0 <= cc < COLS for rr, cc in cells):
                    wins.append(tuple(rr * COLS + cc for rr, cc in cells))
    return wins
WINDOWS = _windows()
COLUMN_ORDER = (1, 2, 0, 3)                    # centre-first move ordering
class Connect3:
    """Immutable state: board is a tuple of 16 cells (row 0 = bottom); X = +1 moves first."""
    __slots__ = ("board", "plies", "winner")
    def __init__(self, board=None, plies=0, winner=0):
        self.board = board if board is not None else (0,) * (ROWS * COLS)
        self.plies, self.winner = plies, winner
    def player(self):
        return 1 if self.plies % 2 == 0 else -1
    def legal_columns(self):
        return [c for c in COLUMN_ORDER if self.board[(ROWS - 1) * COLS + c] == 0]
    def play(self, col):
        b = list(self.board)
        for r in range(ROWS):
            if b[r * COLS + col] == 0:
                b[r * COLS + col] = self.player()
                break
        mover = self.player()
        winner = mover if any(all(b[i] == mover for i in w) for w in WINDOWS) else 0
        return Connect3(tuple(b), self.plies + 1, winner)
    def get_children(self):
        return [self.play(c) for c in self.legal_columns()]
    def is_terminal(self):
        return self.winner != 0 or self.plies == ROWS * COLS
    def evaluate(self):
        """Utility from X's point of view; faster wins score higher."""
        if self.winner:
            return self.winner * (100 - self.plies)
        score = 0
        for w in WINDOWS:
            cells = [self.board[i] for i in w]
            x, o = cells.count(1), cells.count(-1)
            if o == 0:
                score += x * x
            elif x == 0:
                score -= o * o
        return score
```

Listing 4.4: Minimax move selection and the match against a random opponent

```
    def best_move_minimax(state, depth):
    """Choose the column with the highest minimax value for the side to move."""
    me_max = state.player() == 1
    best_v, best_c, counters = None, None, {"nodes_expanded": 0, "eval_calls": 0}
    for col in state.legal_columns():
        v = plain_minimax(state.play(col), depth - 1, not me_max, counters)
        if best_v is None or (v > best_v if me_max else v < best_v):
            best_v, best_c = v, col
    return best_c, counters
def play_game(depth, x_is_engine, rnd):
    s, total_nodes = Connect3(), 0
    while not s.is_terminal():
        engine_turn = (s.player() == 1) == x_is_engine
        if engine_turn:
            col, c = best_move_minimax(s, depth)
            total_nodes += c["nodes_expanded"]
        else:
            col = rnd.choice(s.legal_columns())
        s = s.play(col)
    engine_side = 1 if x_is_engine else -1
    return (1 if s.winner == engine_side else (0.5 if s.winner == 0 else 0)), total_nodes
def experiment_growth():
    print(f"[EXECUTION TRACE - TASK 4 MINIMAX GROWTH (b = {B})]")
    print(f"Prediction (before running): nodes N(d) = ({B}^(d+1)-1)/{B-1} and leaves L(d) = {B}^d; "
          f"time grows ~{B}x per extra ply (i.e. ~{B**2}x per two plies).")
    times = {}
    for d in SYN_DEPTHS:
        c = {"nodes_expanded": 0, "eval_calls": 0}
        t0 = time.perf_counter()
        plain_minimax(SyntheticState(0, B), d, True, c)
        ms = (time.perf_counter() - t0) * 1000
        times[d] = ms
        th_n = (B ** (d + 1) - 1) // (B - 1)
        th_l = B ** d
        print(f"Depth d={d:>2} -> Nodes: {c['nodes_expanded']:>10,} (theory {th_n:,}) | "
              f"Evals: {c['eval_calls']:>10,} (theory {th_l:,}) | Time: {ms:>10.2f} ms")
    ds = sorted(times)
    for a, b_ in zip(ds[1:], ds[:-1]):
        if times[b_] > 0:
            print(f"Time ratio d={a} / d={b_}: {times[a] / times[b_]:.1f}x")
def experiment_match():
    print("\n--- Real game: Connect-3 (4x4) engine of depth d vs a uniformly random opponent ---")
    print("score: win=1, draw=0.5, loss=0")
    rnd = random.Random(SEED)
    for d in GAME_DEPTHS:
        n = GAMES[d]
        wins, score, nodes = 0, 0.0, 0
```

```
        wins, score, nodes = 0, 0.0, 0
        t0 = time.perf_counter()
        for g in range(n):
            x_is_engine = g < n // 2
            r, nn = play_game(d, x_is_engine, rnd)
            score += r
            wins += (r == 1)
            nodes += nn
        ms = (time.perf_counter() - t0) * 1000
        print(f"d={d} | games={n} | wins={wins} | score={100 * score / n:5.1f}% | "
              f"nodes/game={nodes / n:>9,.0f} | ms/game={ms / n:>8.1f}")
# ---------- Experiment 3: duplicate positions ----------
def experiment_duplicates():
    print("\n--- Duplicate positions: distinct states vs states visited by plain Minimax "
          "(Connect-3, empty board) ---")
    for d in DUP_DEPTHS:
        visited, distinct = 0, set()
        def rec(s, depth):
            nonlocal visited
            visited += 1
            distinct.add(s.key())
            if depth == 0 or s.is_terminal():
                return
            for ch in s.get_children():
                rec(ch, depth - 1)
        rec(Connect3(), d)
        red = 100 * (1 - len(distinct) / visited)
        print(f"d={d} | node visits={visited:>8,} | distinct positions={len(distinct):>6,} | "
              f"redundant={red:.1f}%")

if __name__ == "__main__":
    experiment_growth()
    experiment_match()
    experiment_duplicates()
```

4.6 Actual Program Output

Listing 4.5: Execution trace of task4_minimax.py

```
Prediction (before running): nodes N(d) = (4^(d+1)-1)/3 and leaves L(d) = 4^d; time grows ~4x per extra ply (i.e. ~16x per two plies).
Depth d= 2 -> Nodes:         21 (theory 21) | Evals:         16 (theory 16) | Time:       0.06 ms
Depth d= 4 -> Nodes:        341 (theory 341) | Evals:        256 (theory 256) | Time:       0.27 ms
Depth d= 6 -> Nodes:      5,461 (theory 5,461) | Evals:      4,096 (theory 4,096) | Time:       2.62 ms
Depth d= 8 -> Nodes:     87,381 (theory 87,381) | Evals:     65,536 (theory 65,536) | Time:      39.16 ms
Depth d=10 -> Nodes:  1,398,101 (theory 1,398,101) | Evals:  1,048,576 (theory 1,048,576) | Time:     671.91 ms
Time ratio d=4 / d=2: 4.9x
Time ratio d=6 / d=4: 9.8x
Time ratio d=8 / d=6: 14.9x
Time ratio d=10 / d=8: 17.2x

--- Real game: Connect-3 (4x4) engine of depth d vs a uniformly random opponent ---
score: win=1, draw=0.5, loss=0
d=1 | games=40 | wins=37 | score= 92.5% | nodes/game=       14 | ms/game=     0.4
d=2 | games=40 | wins=36 | score= 90.0% | nodes/game=       75 | ms/game=     1.1
d=4 | games=40 | wins=37 | score= 92.5% | nodes/game=      939 | ms/game=    10.2
d=6 | games=40 | wins=39 | score= 97.5% | nodes/game=    9,376 | ms/game=    89.2
d=8 | games=12 | wins=12 | score=100.0% | nodes/game=   80,857 | ms/game=   759.4

--- Duplicate positions: distinct states vs states visited by plain Minimax (Connect-3, empty board) ---
d=2 | node visits=      21 | distinct positions=    21 | redundant=0.0%
d=4 | node visits=     341 | distinct positions=   233 | redundant=31.7%
d=6 | node visits=   4,949 | distinct positions= 1,693 | redundant=65.8%
d=8 | node visits=  58,617 | distinct positions= 7,547 | redundant=87.1%
```

TASK 5 – DERIVE A CONSTRAINT-PROPAGATION ALGORITHM FROM A PRINCIPLE

Source Code Implementation

Listing 5.1: Exam-timetabling CSP generator with a planted solution

```
SIZES      = [10, 20, 30]
SLOTS      = 5
SEEDS      = [1, 2, 3]
EDGE_P     = 0.30
RESTRICT_P = 0.60
NODE_CAP   = 200000
import random
import time
from collections import deque
# ---------- Listing 5.1 ----------
class ExamCSP:
    """Variables = exams, domains = timeslots, binary constraint: clashing exams differ in slot."""
    def __init__(self, n, slots, seed, edge_p=0.30, restrict_p=0.60, infeasible=False):
        rnd = random.Random(seed)
        hidden = [rnd.randrange(slots) for _ in range(n)]
        self.variables = list(range(n))
        self.neighbors = {v: set() for v in self.variables}
        for i in range(n):
            for j in range(i + 1, n):
                if hidden[i] != hidden[j] and rnd.random() < edge_p:
                    self.neighbors[i].add(j); self.neighbors[j].add(i)
        self.domains = {}
        for v in self.variables:
            dom = {hidden[v]}
            for s in range(slots):
                if rnd.random() > restrict_p:
                    dom.add(s)
            self.domains[v] = dom
        if infeasible:
            self.domains[0], self.domains[n - 1] = {0}, {0}
            self.neighbors[0].add(n - 1); self.neighbors[n - 1].add(0)
        self.checks = 0
        self.edges = sum(len(x) for x in self.neighbors.values()) // 2

    def is_consistent(self, x, y):
        self.checks += 1
        return x != y

    def copy_domains(self):
        return {v: set(d) for v, d in self.domains.items()}
```

Listing 5.2: REVISE and AC-3 with a double-ended queue

```
def revise(csp, Xi, Xj):
    revised = False
    for x in set(csp.domains[Xi]):
        if not any(csp.is_consistent(x, y) for y in csp.domains[Xj]):
            csp.domains[Xi].remove(x)
            revised = True
    return revised

def ac3(csp, queue=None):
    """AC-3: returns (consistent?, arc revisions, wipe-out occurred?)."""
    if queue is None:
        queue = deque((Xi, Xj) for Xi in csp.variables for Xj in csp.neighbors[Xi])
    revisions = 0
    while queue:
        Xi, Xj = queue.popleft()
        revisions += 1
        if revise(csp, Xi, Xj):
            if not csp.domains[Xi]:
                return False, revisions, True
            for Xk in csp.neighbors[Xi] - {Xj}:
                queue.append((Xk, Xi))
    return True, revisions, False
```

Listing 5.3: Backtracking search with optional maintenance of arc consistency

```
def backtracking(csp, mode="plain"):
    """mode: 'plain' (check on assignment only) or 'mac' (maintain arc consistency)."""
    stats = {"nodes": 0, "backtracks": 0, "wipeouts": 0, "capped": False}
    order = csp.variables

    def consistent(var, val, assign):
        return all(csp.is_consistent(val, assign[n]) for n in csp.neighbors[var] if n in assign)

    def solve(assign, idx):
        if idx == len(order):
            return dict(assign)
        stats["nodes"] += 1
        if stats["nodes"] > NODE_CAP:
            stats["capped"] = True
            return None
        var = order[idx]
        for val in sorted(csp.domains[var]):
            if not consistent(var, val, assign):
                continue
            assign[var] = val
            if mode == "mac":
                saved = csp.copy_domains()
                csp.domains[var] = {val}
                q = deque((nb, var) for nb in csp.neighbors[var] if nb not in assign)
                ok, _, wiped = ac3(csp, q)
                if ok:
                    res = solve(assign, idx + 1)
                    if res is not None:
                        return res
                if wiped:
                    stats["wipeouts"] += 1
                csp.domains = saved
            else:
                res = solve(assign, idx + 1)
                if res is not None:
                    return res
            del assign[var]
            if stats["capped"]:
                return None
        stats["backtracks"] += 1
        return None
```

```
def is_valid(csp_fresh, sol):
    """Independent check of a returned timetable against the ORIGINAL domains and clashes."""
    if sol is None:
        return False
    for v in csp_fresh.variables:
        if sol[v] not in csp_fresh.domains[v]:
            return False
        for u in csp_fresh.neighbors[v]:
            if sol[u] == sol[v]:
                return False
    return True
def make(n, seed, infeasible=False):
    return ExamCSP(n, SLOTS, seed, EDGE_P, RESTRICT_P, infeasible)
def experiment_ac3():
    print("\nTable A: AC-3 preprocessing")
    print(f"{'n':>3} {'seed':>4} {'edges':>6} {'dom before':>10} {'dom after':>9} "
          f"{'revisions':>9} {'checks':>7} {'consistent':>10}")
    for n in SIZES:
        for s in SEEDS:
            csp = make(n, s)
            before = sum(len(d) for d in csp.domains.values())
            ok, rev, wiped = ac3(csp)
            after = sum(len(d) for d in csp.domains.values())
            print(f"{n:>3} {s:>4} {csp.edges:>6} {before:>10} {after:>9} "
                  f"{rev:>9} {csp.checks:>7} {str(ok):>10}")
def experiment_search(infeasible=False):
    title = "infeasible instances" if infeasible else "feasible instances (planted solution)"
    print(f"\nTable {'C' if infeasible else 'B'}: Backtracking vs MAC on {title}")
    print(f"{'n':>3} {'seed':>4} {'mode':>6} {'nodes':>8} {'backtr':>8} {'wipeouts':>8} "
          f"{'checks':>9} {'ms':>8} {'valid':>6}  result")
    for n in SIZES:
        for s in SEEDS:
            for mode in ("plain", "mac"):
                csp = make(n, s, infeasible)
                orig = make(n, s, infeasible)            # untouched copy for validation
                t0 = time.perf_counter()
                sol, st = backtracking(csp, mode)
                ms = (time.perf_counter() - t0) * 1000
                if st["capped"]:
                    res = "CAPPED"
                elif sol is None:
                    res = "no solution"
                else:
                    res = "solution"
                print(f"{n:>3} {s:>4} {mode:>6} {st['nodes']:>8} {st['backtracks']:>8} "
                      f"{st['wipeouts']:>8} {csp.checks:>9} {ms:>8.1f} "
                      f"{str(is_valid(orig, sol)):>6}  {res}")
```

Actual Program Output

Table A: AC-3 preprocessing

n	seed	edges	dom before	dom after	revisions	checks	consistent
10	1	16	27	27	32	96	True
10	2	6	31	31	12	46	True
10	3	9	26	25	18	53	True
20	1	33	44	38	87	207	True
20	2	51	48	42	135	364	True
20	3	43	46	38	120	275	True
30	1	99	77	40	424	973	True
30	2	106	79	45	444	941	True
30	3	101	80	71	257	730	True

Table B: Backtracking vs MAC on feasible instances (planted solution)

n	seed	mode	nodes	backtr	wipeouts	checks	ms	valid	result
10	1	plain	10	0	0	22	0.0	True	solution
10	1	mac	10	0	0	86	0.1	True	solution
10	2	plain	10	0	0	10	0.0	True	solution
10	2	mac	10	0	0	30	0.1	True	solution
10	3	plain	10	0	0	12	0.0	True	solution
10	3	mac	10	0	0	37	0.1	True	solution
20	1	plain	20	0	0	45	0.1	True	solution
20	1	mac	20	0	0	167	0.2	True	solution
20	2	plain	20	0	0	70	0.1	True	solution
20	2	mac	20	0	0	326	0.3	True	solution
20	3	plain	1052	1032	0	4372	2.2	True	solution
20	3	mac	20	0	3	228	0.9	True	solution
30	1	plain	38587	38557	0	347108	103.1	True	solution
30	1	mac	52	22	32	3452	4.1	True	solution
30	2	plain	5898	5868	0	53061	20.9	True	solution
30	2	mac	30	0	1	686	0.9	True	solution
30	3	plain	70756	70726	0	492095	162.5	True	solution
30	3	mac	53	23	36	4954	3.2	True	solution

Table C: Backtracking vs MAC on infeasible instances

n	seed	mode	nodes	backtr	wipeouts	checks	ms	valid	result
10	1	plain	24	24	0	72	0.0	False	no solution
10	1	mac	1	1	1	8	0.0	False	no solution
10	2	plain	4865	4865	0	7842	5.5	False	no solution
10	2	mac	1	1	1	1	0.0	False	no solution
10	3	plain	791	791	0	2143	1.2	False	no solution
10	3	mac	1	1	1	7	0.0	False	no solution
20	1	plain	2136	2136	0	8548	3.1	False	no solution
20	1	mac	1	1	1	3	0.0	False	no solution
20	2	plain	12231	12231	0	85104	24.4	False	no solution
20	2	mac	1	1	1	17	0.0	False	no solution
20	3	plain	2075	2075	0	4590	2.9	False	no solution
20	3	mac	1	1	1	8	0.0	False	no solution
30	1	plain	38246	38246	0	338512	106.9	False	no solution
30	1	mac	1	1	1	4	0.0	False	no solution
30	2	plain	65521	65521	0	552369	210.0	False	no solution
30	2	mac	1	1	1	8	0.0	False	no solution
30	3	plain	43212	43212	0	282116	140.7	False	no solution
30	3	mac	1	1	1	18	0.1	False	no solution

TASK 6 – EXHAUSTIVE SEARCH, LOCAL SEARCH AND COMBINATORIAL GROWTH

Source Code Implementation

Listing 6.1: Random instance generator and tour-length function

```
SMALL_N    = [6, 7, 8, 9, 10]
LARGE_N    = [20, 50, 100]
SEEDS      = [1, 2, 3]
RESTARTS   = 10
import itertools
import math
import random
import time
import numpy as np
# ---------- Listing 6.1 ----------
def make_instance(n, seed):
    """n random customer locations in a 100 x 100 km area; Euclidean distance matrix."""
    rng = np.random.default_rng(seed)
    pts = rng.uniform(0, 100, size=(n, 2))
    d = np.sqrt(((pts[:, None, :] - pts[None, :, :]) ** 2).sum(-1))
    return pts, d.tolist()

def tour_cost(dist, tour):
    return sum(dist[tour[i]][tour[i + 1]] for i in range(len(tour) - 1))
```

Listing 6.2: Exhaustive enumeration of all distinct tours

```
# ---------- Listing 6.2 ----------
def exhaustive_tsp(dist):
    """Enumerate every distinct tour: city 0 fixed, mirror images counted once -> (n-1)!/2 tours."""
    n = len(dist)
    min_cost, best_tour, count = float("inf"), None, 0
    t0 = time.perf_counter()
    for perm in itertools.permutations(range(1, n)):
        if perm[0] > perm[-1]:                    # skip the reversed duplicate of a tour
            continue
        count += 1
        cost = dist[0][perm[0]] + dist[perm[-1]][0]
        for i in range(n - 2):
            cost += dist[perm[i]][perm[i + 1]]
        if cost < min_cost:
            min_cost, best_tour = cost, [0] + list(perm) + [0]
    return min_cost, best_tour, count, time.perf_counter() - t0
```

Listing 6.3: 2-Opt local search with constant-time move evaluation and random restarts

```
# ---------- Listing 6.3 ----------
def two_opt_once(dist, tour, counters):
    """Repeat improving 2-opt moves until a local optimum is reached."""
    n = len(tour) - 1
    improved = True
    while improved:
        improved = False
        for i in range(1, n - 1):
            for j in range(i + 1, n):
                counters["evaluated"] += 1
                a, b, c, d = tour[i - 1], tour[i], tour[j], tour[j + 1]
                delta = dist[a][c] + dist[b][d] - dist[a][b] - dist[c][d]
                if delta < -1e-12:
                    tour[i:j + 1] = tour[i:j + 1][::-1]
                    counters["moves"] += 1
                    improved = True
    return tour

def two_opt_tsp(dist, restarts=10, seed=0):
    n = len(dist)
    rnd = random.Random(seed)
    best_cost, best_tour = float("inf"), None
    counters = {"evaluated": 0, "moves": 0}
    t0 = time.perf_counter()
    for _ in range(restarts):
        perm = list(range(1, n))
        rnd.shuffle(perm)
        tour = two_opt_once(dist, [0] + perm + [0], counters)
        cost = tour_cost(dist, tour)
        if cost < best_cost:
            best_cost, best_tour = cost, tour[:]
    return best_cost, best_tour, counters, time.perf_counter() - t0

# ---------- Experiment driver ----------
def valid_tour(tour, n):
    return tour[0] == 0 and tour[-1] == 0 and sorted(tour[:-1]) == list(range(n))

def experiment_small():
    print("\nTable A: Exhaustive vs 2-Opt (small n)")
    print(f"{'n':>3} {'seed':>4} {'tours':>9} {'(n-1)!/2':>9} {'opt cost':>9} {'2opt cost':>9} "
          f"{'gap%':>6} {'t_exh(s)':>9} {'t_2opt(s)':>9} {'evals':>7} {'moves':>6} {'valid':>6}")
    for n in SMALL_N:
        for s in SEEDS:
            _, dist = make_instance(n, s)
            oc, ot, cnt, te = exhaustive_tsp(dist)
            c2, t2, ctr, t2s = two_opt_tsp(dist, RESTARTS, s)
            gap = 100 * (c2 - oc) / oc
            ok = valid_tour(ot, n) and valid_tour(t2, n) and abs(tour_cost(dist, t2) - c2) < 1e-9
            print(f"{n:>3} {s:>4} {cnt:>9} {math.factorial(n - 1) // 2:>9} {oc:>9.2f} {c2:>9.2f} "
                  f"{gap:>6.2f} {te:>9.3f} {t2s:>9.4f} {ctr['evaluated']:>7} {ctr['moves']:>6} {str(ok):>6}")

def experiment_large():
    print("\nTable B: 2-Opt on larger instances (exhaustive infeasible)")
    print(f"{'n':>4} {'seed':>4} {'best cost':>10} {'time(s)':>8} {'evals':>9} {'moves':>7} {'valid':>6}")
    for n in LARGE_N:
        for s in SEEDS:
            _, dist = make_instance(n, s)
            c2, t2, ctr, ts = two_opt_tsp(dist, RESTARTS, s)
            print(f"{n:>4} {s:>4} {c2:>10.2f} {ts:>8.3f} {ctr['evaluated']:>9} "
                  f"{ctr['moves']:>7} {str(valid_tour(t2, n)):>6}")

def plot_tour(n=LARGE_N[0], seed=SEEDS[0]):
    import matplotlib.pyplot as plt
    pts, dist = make_instance(n, seed)
    c, t, _, _ = two_opt_tsp(dist, RESTARTS, seed)
    xs, ys = [pts[i][0] for i in t], [pts[i][1] for i in t]
    plt.plot(xs, ys, "-o", markersize=4)
    plt.plot(pts[0][0], pts[0][1], "rs", markersize=9, label="depot")
    plt.title(f"2-Opt tour (n={n}, seed={seed}, cost={c:.1f} km)")
    plt.xlabel("x (km)"); plt.ylabel("y (km)"); plt.legend(); plt.show()

if __name__ == "__main__":
    experiment_small()
    experiment_large()
    plot_tour()
```

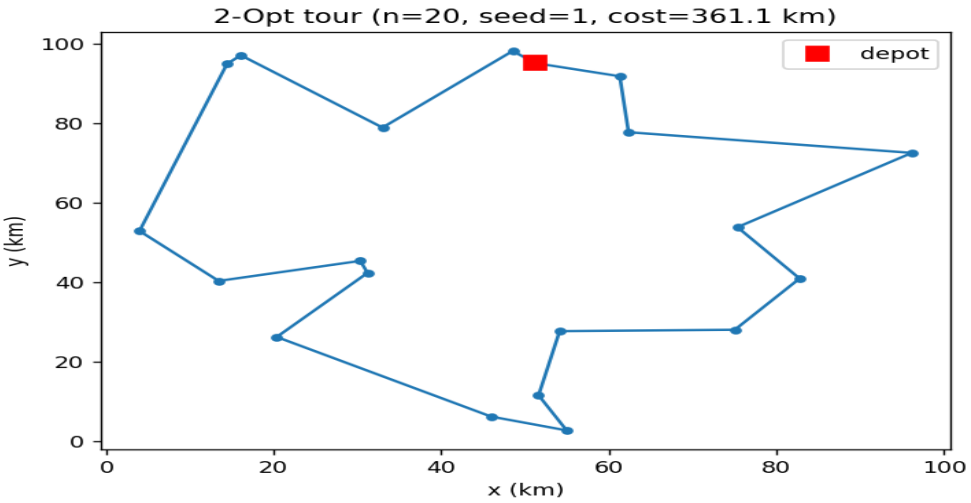
Actual Program Output

Table A: Exhaustive vs 2-Opt (small n)

n	seed	tours	(n-1)!/2	opt cost	2opt cost	gap%	t_exh(s)	t_2opt(s)	evals	moves	valid
6	1	60	60	247.97	247.97	0.00	0.000	0.0001	200	28	True
6	2	60	60	226.28	226.28	-0.00	0.000	0.0001	250	33	True
6	3	60	60	205.41	205.41	0.00	0.000	0.0002	210	30	True
7	1	360	360	253.90	253.90	-0.00	0.000	0.0002	375	54	True
7	2	360	360	226.57	226.57	-0.00	0.000	0.0002	375	49	True
7	3	360	360	208.99	208.99	0.00	0.000	0.0002	390	45	True
8	1	2520	2520	254.08	254.08	-0.00	0.002	0.0002	588	68	True
8	2	2520	2520	229.48	229.48	-0.00	0.002	0.0002	567	55	True
8	3	2520	2520	257.92	257.92	0.00	0.002	0.0002	483	49	True
9	1	20160	20160	280.30	280.30	0.00	0.010	0.0003	1064	106	True
9	2	20160	20160	260.39	260.39	-0.00	0.011	0.0002	644	74	True
9	3	20160	20160	272.76	272.76	0.00	0.011	0.0002	840	78	True
10	1	181440	181440	283.33	283.33	0.00	0.135	0.0004	1296	139	True
10	2	181440	181440	263.52	263.52	0.00	0.114	0.0002	864	91	True
10	3	181440	181440	274.55	274.55	0.00	0.133	0.0004	1008	84	True

Table B: 2-Opt on larger instances (exhaustive infeasible)

n	seed	best cost	time(s)	evals	moves	valid
20	1	361.07	0.002	6498	345	True
20	2	372.37	0.001	5985	302	True
20	3	395.54	0.001	5814	290	True
50	1	535.94	0.012	48216	1156	True
50	2	595.45	0.016	59976	1128	True
50	3	576.82	0.010	56448	1188	True
100	1	795.01	0.063	257103	2914	True
100	2	832.09	0.069	291060	2908	True
100	3	829.01	0.077	257103	2973	True



TASK 7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE

Listing 7.1: Alpha-Beta with a transposition table and bound flags (the redesign)

```
DEPTHS        = [4, 6, 8]
N_POSITIONS   = 12
POS_SEED      = 2026
MAX_RANDOM_PLIES = 6
STRESS_N      = 200
STRESS_DEPTH  = 6
STRESS_SEED   = 7
EMPTY_DEPTH   = 8
import random
import time
INF = float("inf")
ROWS, COLS, K = 4, 4, 3
# ---------- Connect-3 (Listing 4.3) ----------
def _windows():
    wins = []
    for r in range(ROWS):
        for c in range(COLS):
            for dr, dc in ((0, 1), (1, 0), (1, 1), (1, -1)):
                cells = [(r + i * dr, c + i * dc) for i in range(K)]
                if all(0 <= rr < ROWS and 0 <= cc < COLS for rr, cc in cells):
                    wins.append(tuple(rr * COLS + cc for rr, cc in cells))
    return wins
WINDOWS = _windows()
COLUMN_ORDER = (1, 2, 0, 3)
class Connect3:
    __slots__ = ("board", "plies", "winner")
    def __init__(self, board=None, plies=0, winner=0):
        self.board = board if board is not None else (0,) * (ROWS * COLS)
        self.plies, self.winner = plies, winner
    def player(self):
        return 1 if self.plies % 2 == 0 else -1
    def legal_columns(self):
        return [c for c in COLUMN_ORDER if self.board[(ROWS - 1) * COLS + c] == 0]
    def play(self, col):
        b = list(self.board)
        for r in range(ROWS):
            if b[r * COLS + col] == 0:
                b[r * COLS + col] = self.player()
                break
        mover = self.player()
        winner = mover if any(all(b[i] == mover for i in w) for w in WINDOWS) else 0
        return Connect3(tuple(b), self.plies + 1, winner)
    def get_children(self):
        return [self.play(c) for c in self.legal_columns()]
    def is_terminal(self):
        return self.winner != 0 or self.plies == ROWS * COLS
    def evaluate(self):
```

```
        if self.winner:
            return self.winner * (100 - self.plies)
        score = 0
        for w in WINDOWS:
            cells = [self.board[i] for i in w]
            x, o = cells.count(1), cells.count(-1)
            if o == 0:
                score += x * x
            elif x == 0:
                score -= o * o
        return score
    def key(self):
        return self.board
```

```python
def plain_minimax(state, depth, is_max, counters):
    counters["nodes_expanded"] += 1
    if depth == 0 or state.is_terminal():
        counters["eval_calls"] += 1
        return state.evaluate()
    if is_max:
        best = -INF
        for child in state.get_children():
            best = max(best, plain_minimax(child, depth - 1, False, counters))
        return best
    best = INF
    for child in state.get_children():
        best = min(best, plain_minimax(child, depth - 1, True, counters))
    return best
```

```python
# ---------- Listing 7.1 ----------
EXACT, LOWER, UPPER = 0, 1, 2

def alphabeta_tt(state, depth, alpha, beta, maximizing, c, table):
    c["nodes"] += 1
    alpha0, beta0 = alpha, beta
    key = state.key()
    entry = table.get(key)
    if entry is not None and entry[0] >= depth:
        _, val, flag = entry
        if flag == EXACT:
            c["hits"] += 1
            return val
        if flag == LOWER:
            alpha = max(alpha, val)
        else:
            beta = min(beta, val)
        if alpha >= beta:
            c["hits"] += 1
            return val
    if depth == 0 or state.is_terminal():
        c["evals"] += 1
        return state.evaluate()
    if maximizing:
        best = -INF
        for child in state.get_children():
            best = max(best, alphabeta_tt(child, depth - 1, alpha, beta, False, c, table))
            alpha = max(alpha, best)
            if alpha >= beta:
                c["cutoffs"] += 1
                break
    else:
        best = INF
        for child in state.get_children():
            best = min(best, alphabeta_tt(child, depth - 1, alpha, beta, True, c, table))
            beta = min(beta, best)
            if alpha >= beta:
                c["cutoffs"] += 1
                break
    flag = EXACT
    if best <= alpha0:
        flag = UPPER
    elif best >= beta0:
        flag = LOWER
    if entry is None or entry[0] <= depth:
        table[key] = (depth, best, flag)
    return best
```

Listing 7.2: Root move selection shared by the three engines

```
# ---------- Listing 7.2 ----------
def root_choice(state, depth, engine):
    maximizing = state.player() == 1
    c = {"nodes": 0, "evals": 0, "cutoffs": 0, "hits": 0}
    table = {}
    alpha, beta = -INF, INF
    best_v, best_col = None, None
    for col in state.legal_columns():
        child = state.play(col)
        if engine == "minimax":
            cc = {"nodes_expanded": 0, "eval_calls": 0}
            v = plain_minimax(child, depth - 1, not maximizing, cc)
            c["nodes"] += cc["nodes_expanded"]; c["evals"] += cc["eval_calls"]
        elif engine == "ab":
            v = alphabeta(child, depth - 1, alpha, beta, not maximizing, c)
        else:
            v = alphabeta_tt(child, depth - 1, alpha, beta, not maximizing, c, table)
        if best_v is None or (v > best_v if maximizing else v < best_v):
            best_v, best_col = v, col
        if engine != "minimax":
            alpha, beta = (max(alpha, best_v), beta) if maximizing else (alpha, min(beta, best_v))
    c["table_size"] = len(table)
    return best_v, best_col, c
def random_position(rnd):
    """Random 0..MAX_RANDOM_PLIES plies by random play from the empty board; non-terminal only."""
    while True:
        s = Connect3()
        for _ in range(rnd.randint(0, MAX_RANDOM_PLIES)):
            if s.is_terminal():
                break
            s = s.play(rnd.choice(s.legal_columns()))
        if not s.is_terminal():
            return s
def timed(state, depth, engine):
    t0 = time.perf_counter()
    r = root_choice(state, depth, engine)
    return r, (time.perf_counter() - t0) * 1000
def experiment_validation():
    print("[EXECUTION TRACE - TASK 7 REDESIGN VALIDATION]")
    print(f"Test set: {N_POSITIONS} reproducible Connect-3 (4x4) positions generated with seed {POS_SEED}.\n")
    rnd = random.Random(POS_SEED)
    positions = [random_position(rnd) for _ in range(N_POSITIONS)]
    for d in DEPTHS:
        tot = {e: {"evals": 0, "cutoffs": 0, "hits": 0, "ms": 0.0} for e in ("minimax", "ab", "tt")}
        same_v = same_m = 0
        for p in positions:
            res = {}

            for e in ("minimax", "ab", "tt"):
                (v, col, c), ms = timed(p, d, e)
                res[e] = (v, col)
                tot[e]["evals"] += c["evals"]; tot[e]["cutoffs"] += c["cutoffs"]
                tot[e]["hits"] += c["hits"]; tot[e]["ms"] += ms
            same_v += (res["minimax"][0] == res["ab"][0] == res["tt"][0])
            same_m += (res["minimax"][1] == res["ab"][1] == res["tt"][1])
        mm, ab, tt = tot["minimax"], tot["ab"], tot["tt"]
        pct = lambda a, b: 100 * (1 - a / b)
        print(f"Depth d={d}: minimax evals={mm['evals']:>9,} | alpha-beta evals={ab['evals']:>7,} "
              f"({pct(ab['evals'], mm['evals']):.1f}% fewer) | AB+TT evals={tt['evals']:>7,} "
              f"({pct(tt['evals'], mm['evals']):.1f}% fewer)")
        print(f"cutoffs(AB)={ab['cutoffs']:,} cutoffs(TT)={tt['cutoffs']:,} cache hits={tt['hits']:,} | "
              f"time MM={mm['ms']:.1f} ms AB={ab['ms']:.1f} ms TT={tt['ms']:.1f} ms "
              f"({pct(tt['ms'], mm['ms']):.1f}% less)")
        print(f"same root value: {same_v}/{len(positions)} | same chosen move: {same_m}/{len(positions)}\n")

def experiment_empty():
    print(f"--- Empty board, depth {EMPTY_DEPTH} (single position) ---")
    for e in ("minimax", "ab", "tt"):
        (v, col, c), ms = timed(Connect3(), EMPTY_DEPTH, e)
        print(f"{e:<8} value={v} move={col} evals={c['evals']:,} nodes={c['nodes']:,} "
              f"hits={c['hits']} cutoffs={c['cutoffs']}
```

Actual program output

```
Test set: 12 reproducible Connect-3 (4x4) positions generated with seed 2026.

Depth d=4: minimax evals=    2,270 | alpha-beta evals=    608 (73.2% fewer) | AB+TT evals=    576 (74.6% fewer)
cutoffs(AB)=262 cutoffs(TT)=234 cache hits=29 | time MM=30.5 ms AB=14.5 ms TT=13.7 ms (55.1% less)
same root value: 12/12 | same chosen move: 12/12

Depth d=6: minimax evals=   24,260 | alpha-beta evals=  2,536 (89.5% fewer) | AB+TT evals=  1,950 (92.0% fewer)
cutoffs(AB)=1,462 cutoffs(TT)=1,065 cache hits=265 | time MM=355.9 ms AB=68.6 ms TT=59.6 ms (83.3% less)
same root value: 12/12 | same chosen move: 12/12

Depth d=8: minimax evals=  200,714 | alpha-beta evals=  5,251 (97.4% fewer) | AB+TT evals=  3,359 (98.3% fewer)
cutoffs(AB)=3,564 cutoffs(TT)=2,282 cache hits=797 | time MM=2783.7 ms AB=131.6 ms TT=104.0 ms (96.3% less)
same root value: 12/12 | same chosen move: 12/12

--- Empty board, depth 8 (single position) ---
minimax  value=0 move=1 evals=43,104 nodes=58,616 hits=0 cutoffs=0 table=0 time=621.4 ms
ab       value=0 move=1 evals=894 nodes=1,776 hits=0 cutoffs=651 table=0 time=23.1 ms
tt       value=0 move=1 evals=555 nodes=1,275 hits=136 cutoffs=407 table=569 time=16.6 ms

--- Correctness stress test: 200 random positions, depth 6, value equality ---
Mismatches between plain Minimax and AB+TT: 0 of 200 positions
```